Testing multiple_table.sql locally

CROSSFADE, info579_group_1 (Herling, Nathan; Pabari, Nidhi Nilesh). INFO 579 Project 2, Task 3. 80 tests; last run 2026-11-07 22:41:15: 80 passed, 0 failed.

How multiple_table.sql is checked: every query is run on the crossfade database and compared with an answer computed independently in Python from data.csv; MySQL's own plan proves which tables and columns it reads; the SQL must really meet the Task 3 requirement its comment claims, with every join named; and joins, boundaries and NULLs the sample data never reaches are tested by adding rows in rolled-back transactions.

1. What the tests compare against

Source	From	How it is used
Independent answers	oracle_multiple.py + data.csv	Each query's answer is recomputed in plain Python from data.csv, no SQL and no database. MySQL must return the same columns, rows and order, to the exact decimal.
The use cases	use_cases.json ← requirements_analysis.pdf	Each query names the use cases it answers; each use case quotes the requirements analysis word for word and lists, under multipleColumns, the columns its query reads.
MySQL's own plan	EXPLAIN FORMAT=JSON	The tables each query reads and the columns it uses from each (used_columns), from MySQL's execution plan rather than a text search of the SQL.
The live schema	information_schema	Which tables are associative and which tables they relate (3.1), from the foreign keys database_implementation.sql created.
The brief and the slides	info579_project02.pdf, Units 8 and 9	Requirements 3.1-3.5 from the brief; set operations, multi-row operators and WITH from Unit 8; named join types (no bare JOIN, no comma join) from Unit 9.

2. Run them

a. What you need. A running MySQL Server 8.0.16 or later and its root password; Python 3.10 or later. The mysql command-line client and Workbench are not needed: --fresh builds the database for you.

b. Get the files. Download crossfade_multiple_tests.zip with the button above and extract it. It holds one folder, crossfade_multiple_tests, with the tests, the oracle, multiple_table.sql, database_implementation.sql, data.csv, use_cases.json and the guide.

```
Expand-Archive -Force $HOME\Downloads\crossfade_multiple_tests.zip .
cd .\crossfade_multiple_tests
```

c. Set up Python (once). A virtual environment with the three packages. macOS / Linux: python3 and .venv/bin/python.

```
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
```

d. Run the tests. --fresh rebuilds the crossfade database from database_implementation.sql first. Without it the tests use the database as it is, after checking it holds exactly the rows in data.csv. Every write a test makes is rolled back.

```
$env:CROSSFADE_DB_PASSWORD = "your-root-password"
.venv\Scripts\python -m pytest --fresh
```

e. Read the result. One line per test, then a summary. A correctness failure prints the first row where MySQL and the oracle differ; a contract failure names the comment field, table, column or join. Each query's result as the tests saw it is on the multiple_table page (Project 2 → Files) and on the Use cases page.

```
collected 80 items
test_multiple_table.py::test_comment_contract[M1] PASSED   [  1%]
...
========== 80 passed in 0.3s ==========
```

f. Write the queries yourself. The starter kit (Project 2 → single_table → Download starter kit) holds multiple_table.sql with every comment kept and every query removed. Run the tests at any point: unwritten queries are reported by name.

3. What each group proves

Group	Tests	What it proves
contract	42/42	Each query is preceded by a purpose comment with its tag, title, use cases, business need, business question, the Task 3 requirement it meets, the tables it reads, what one result row is and its output columns. It is one read-only SELECT (or WITH ... SELECT) with no SELECT *. MySQL's own plan (EXPLAIN) reads exactly the tables the comment lists. The SQL really meets the requirement it claims (3.1 an associative table INNER JOINed to the two tables it relates, found from the live foreign keys; 3.2 LEFT OUTER JOIN; 3.3 a set operation; 3.4 a multi-row subquery operator; 3.5 WITH). Every join names its type. Every column MySQL reads is one its use cases declare.
correctness	14/14	Each result equals an answer computed independently in plain Python from data.csv (oracle_multiple.py: joins are dictionary look-ups, UNION is a set, WITH is a list built first): same columns, rows, order, exact decimals. There is one row per declared grain, the ORDER BY includes every grain column so no two rows can tie, and two runs agree.
behaviour	20/20	Logic the sample data never reaches: rows are added inside a transaction that is rolled back, and the query must still agree with the oracle on the changed data. Each case targets one join, boundary, NULL or tie.
coverage	4/4	The file as a whole: each of the five Task 3 requirements is met by a query that claims it; every use case with multiple-table columns is answered, and no query names one without them; every column a use case declares is read by a query answering it (two queries can answer one use case: M4 and M7 both answer UC-11); tags run M1, M2, ... in order.

4. Files

File	Role
multiple_table.sql	the file under test (Task 3)
test_multiple_table.py	the tests: contract, correctness, behaviour cases, file coverage
oracle_multiple.py	the independent answers: each query recomputed in plain Python from data.csv, no SQL
oracle.py	shared helpers the multiple-table oracle imports (MySQL-style rounding)
conftest.py	shared set-up: finds the files, connects, splits the .sql into commented statements, writes the results snapshot
use_cases.json	the use cases: their RA evidence and the columns each multiple-table query may read (multipleColumns)
data.csv	the Project 1 sample data the oracle reads
database_implementation.sql	builds the database the queries run on (--fresh)
pytest.ini	pytest settings (verbose, no cache folder)
requirements.txt	pytest, PyMySQL, cryptography (MySQL 8 sign-in)
test_multiple_table_guide.pdf	how to run the tests locally, what they cover and what they could miss
mutate_multiple_table.py	tests the tests: breaks multiple_table.sql 39 ways, one at a time, and checks every break is caught (repo only)
crossfade_multiple_tests.zip	the tests, the oracle, every input file and the guide, in one folder that runs as-is

5. What the tests could miss

Passing means the file agrees with its sources and refuses what they forbid. It does not mean the following:

Gap	What could slip through	What covers it instead
Independent code, not an independent reading	The oracle was written from the same comments as the queries. If a business rule was misread (what 'reach' means, which plays qualify), the query and its expected answer agree and the test passes.	Each comment cites the requirements analysis section by section; a second reader.
A small sample	8 artists, 15 tracks, 8 listeners, 8 stations, 27 scores, 270 probabilities, 31 plays. Some joins never lose a row and some boundaries are never reached; they are tested only where a behaviour case adds the row.	The 20 behaviour cases, and the deliberate-mistake run below.
Which version is 'in service'	M6 reads 'a model redeployment' (RA: Scored On) as a newer version going into service: the one deployed last. If the business meant redeploying the same version (its deployed_on moving later), the same comparison applies to that version's own results, and M6 would need a second condition.	The reading is stated in M6's comment; a behaviour case deploys a new version and checks every recording becomes due.
Two versions always agree on divergence	Every disagreement in the sample is between exactly two model versions, whose divergences from each other are the same number, so MAX and MIN give the same answer on the sample data.	A behaviour case adds a third version with a higher divergence (found by the mistake run).
Requirements recognised by shape, not by merit	3.1-3.5 are detected from the SQL (join types, UNION, IN (SELECT, WITH). Whether a query is 'trivial or generic' (the brief's words) is the grader's judgement, which no test can make.	Each comment says why the join, set operation or subquery is needed for the business question.
Comments checked for shape, not truth	Every comment field must be present and agree with the SQL (tables, grain, output, requirement). Whether the business reasoning is sound is not testable.	The comments cite the requirements analysis.
Our own column lists	'Columns as declared' is checked against multipleColumns in use_cases.json, which we wrote. A wrong list there would pass a query that reads the wrong column.	build_usecases.py checks each multipleColumns entry is a column the use case needs and exists in the model.
A plan-dependent column check	used_columns comes from MySQL's optimizer. A different MySQL version could list a column differently (an index it chose), and the strict both-ways check would then fail on a correct query.	Only columns the statement names count, which drops the index-only extras; verified on MySQL 8.0.
Speed	No plan is judged for cost (M4's correlated subqueries run once per recording). The tables are tiny.	Not required by the brief.

6. Testing the tests

mutate_multiple_table.py copies the file, makes one deliberate mistake, and runs every test against the copy. A mistake is caught when a test fails that passes on the real file. 39 of 39 were caught.

Mistake	Caught	First failing tests
listener joined with LEFT OUTER JOIN (3.1 asks for INNER JOIN on all three tables)	yes (1 failed)	test_meets_its_requirement[M1]
share heard as the average of per-play ratios, not total heard / total length	yes (3 failed)	test_result_matches_oracle[M1], test_m1_listener_with_no_plays_is_absent, test_m1_abandoned_skip_lowers_the_share_heard
least engaged listeners first	yes (3 failed)	test_result_matches_oracle[M1], test_m1_listener_with_no_plays_is_absent, test_m1_abandoned_skip_lowers_the_share_heard
the listener_id tie-break left out of ORDER BY	yes (1 failed)	test_result_is_deterministic[M1]
track left off the comment's Tables line	yes (1 failed)	test_reads_declared_tables[M1]
play joined with INNER JOIN: artists with no plays drop out of the royalty run	yes (3 failed)	test_result_matches_oracle[M2], test_m2_artist_with_uploads_but_no_plays_is_owed_zero, test_m2_short_play_is_counted_but_does_not_qualify
IFNULL removed: an artist with no plays is owed NULL instead of 0	yes (3 failed)	test_result_matches_oracle[M2], test_m2_artist_with_uploads_but_no_plays_is_owed_zero, test_m2_short_play_is_counted_but_does_not_qualify

Mistake	Caught	First failing tests
COUNT(*) instead of COUNT(p.track_id): an artist with no plays shows 1 play	yes (3 failed)	test_result_matches_oracle[M2], test_m2_artist_with_uploads_but_no_plays_is_owed_zero, test_m2_short_play_is_counted_but_does_not_qualify
a play of exactly 30 seconds does not qualify (> instead of >=)	yes (1 failed)	test_m2_short_play_is_counted_but_does_not_qualify
LEFT JOIN written without OUTER (the course asks for the full join type)	yes (1 failed)	test_join_types_explicit[M2]
a column the use cases do not declare is read (artist.country)	yes (1 failed)	test_columns_match_use_cases[M2]
UNION ALL: a listener who plays a station twice is counted twice	yes (3 failed)	test_result_matches_oracle[M3], test_m3_listener_playing_anothers_station_shares_it, test_m3_unplayed_station_still_reaches_its_creator
a station reaching only its creator labelled shared (>= 1 instead of > 1)	yes (3 failed)	test_result_matches_oracle[M3], test_m3_listener_playing_anothers_station_shares_it, test_m3_unplayed_station_still_reaches_its_creator
a bare JOIN with no join type	yes (1 failed)	test_join_types_explicit[M3]
the station_id tie-break left out of ORDER BY	yes (1 failed)	test_result_is_deterministic[M3]
the comment says one row per creator (a creator can have two stations)	yes (1 failed)	test_result_is_deterministic[M3]
two scores of any genre counted as a disagreement (COUNT(*) instead of COUNT(DISTINCT genre_id))	yes (5 failed)	test_result_matches_oracle[M4], test_m4_new_disagreeing_score_adds_the_recording, test_m4_third_version_raises_the_highest_divergence
NOT IN: the recordings the models agree on	yes (5 failed)	test_result_matches_oracle[M4], test_m4_new_disagreeing_score_adds_the_recording, test_m4_third_version_raises_the_highest_divergence
the lowest divergence instead of the highest	yes (1 failed)	test_m4_third_version_raises_the_highest_divergence
the track_id tie-break left out of ORDER BY	yes (1 failed)	test_result_is_deterministic[M4]
the comment claims requirement 3.3 (a set operation) instead of 3.4	yes (2 failed)	test_meets_its_requirement[M4], test_each_requirement_is_met
the window's edges left out (> and < instead of BETWEEN)	yes (1 failed)	test_m5_window_edges_are_inside
rejected recordings allowed on stations (ingest status not checked)	yes (2 failed)	test_m5_rejected_recording_is_never_streamed, test_declared_columns_are_read
scores from every model version used, not the station's own	yes (6 failed)	test_result_matches_oracle[M5], test_result_is_deterministic[M5], test_m5_window_edges_are_inside
top genre not matched to the blend (any blend row joins)	yes (5 failed)	test_result_matches_oracle[M5], test_result_is_deterministic[M5], test_m5_window_edges_are_inside
lowest blend weight first	yes (4 failed)	test_result_matches_oracle[M5], test_m5_window_edges_are_inside, test_m5_rejected_recording_is_never_streamed
the track_id tie-break left out of ORDER BY	yes (1 failed)	test_result_is_deterministic[M5]
station running time not rounded	yes (4 failed)	test_result_matches_oracle[M5], test_m5_window_edges_are_inside, test_m5_rejected_recording_is_never_streamed
a result scored at midnight on the deployment day counted as out of date (<= instead of <)	yes (1 failed)	test_m6_deployment_day_is_the_boundary
the oldest result judged instead of the newest: rescored recordings stay listed	yes (4 failed)	test_result_matches_oracle[M6], test_m6_rescore_by_the_model_in_service_brings_a_recording_up_to_date, test_m6_deployment_day_is_the_boundary

Mistake	Caught	First failing tests
rejected recordings queued for rescoring (release status not checked)	yes (2 failed)	test_m6_rejected_recording_is_never_rescored, test_declared_columns_are_read
days counted from the first deployment instead of the one in service	yes (5 failed)	test_result_matches_oracle[M6], test_m6_rescore_by_the_model_in_service_brings_a_recording_up_to_date, test_m6_deployment_day_is_the_boundary
the versions listed alphabetically instead of oldest first	yes (5 failed)	test_result_matches_oracle[M6], test_m6_rescore_by_the_model_in_service_brings_a_recording_up_to_date, test_m6_deployment_day_is_the_boundary
the track_id tie-break left out of ORDER BY	yes (1 failed)	test_result_is_deterministic[M6]
the top genre allowed as its own runner-up: every margin is 0	yes (4 failed)	test_result_matches_oracle[M7], test_m7_tie_for_second_place_takes_the_lower_genre_id, test_m7_top_tie_is_exactly_between_genres
a tie for second place goes to the higher genre_id	yes (1 failed)	test_m7_tie_for_second_place_takes_the_lower_genre_id
the most clearly placed recordings first	yes (4 failed)	test_result_matches_oracle[M7], test_m7_tie_for_second_place_takes_the_lower_genre_id, test_m7_top_tie_is_exactly_between_genres
the model_id tie-break left out of ORDER BY	yes (1 failed)	test_result_is_deterministic[M7]
results with no distribution kept, half empty (runner_up and its genre joined LEFT OUTER)	yes (1 failed)	test_m7_result_without_a_distribution_is_absent

7. Findings

The tests had a blind spot (now fixed). The deliberate-mistake run showed that reporting the lowest divergence instead of the highest in [M4] went unnoticed: every listed sample recording is scored by exactly two versions, whose divergences from each other are equal. A behaviour case now adds a third version with a higher divergence.